

Оценка проекта первой группы по соответствию ТЗ и задачам спринта S1

☒ Что выполнено и соответствует ТЗ и S1:

1. Базовая структура проекта

- ☒ Присутствуют папки `app/`, `alembic/`, `requirements.txt`, `database.py`, `models.py`, `schemas.py`, `.ini`, `main.py` — структура соответствует задаче [S1] Создать репозиторий и структуру проекта.

2. Настройка зависимостей

- ☒ В `requirements.txt` указаны все библиотеки из ТЗ: FastAPI, SQLAlchemy 2.x, Alembic, Pydantic 2.x, asyncpg, bcrypt, jose, FastStream и др.
- ☐ ⚠ Названия некоторых функций вызывают сомнение (`creat_access_token` → `create_access_token`, `oauth2_sheme` → `oauth2_scheme`).

3. Модели пользователей и постов

- ☒ Реализованы модели `User` и `Post` с нужными полями и связью один-ко-многим.
- ☒ Используется SQLAlchemy 2.x + `Base` вынесен в отдельный модуль (`db_base.py`).
- ☒ Миграции сгенерированы и применены через Alembic (`alembic/versions/*.py`) — соответствует [S1] Реализовать модели пользователя и поста.

4. Регистрация и авторизация

- ☒ `register_user()` содержит критичную ошибку: хэшируется `exist_user.password`, а не `user_in.password`.
 - `exist_user` — это **результат запроса к БД**, чтобы проверить, существует ли уже пользователь с таким email:

```
exist_user = await
db.execute(select(models.User).filter(models.User.email ==
user_in.email))
existing_user = exist_user.scalar_one_or_none()
```

- Но `exist_user.password` вообще **не существует** — `exist_user` — это **ResultProxy**, а не `User`.
- Даже если бы был `existing_user`, то в случае, когда пользователь **ещё не зарегистрирован**, `existing_user` будет `None`.

★ Получается, вы хешируете **несуществующее поле у объекта, которого нет**, а не то, что ввёл пользователь при регистрации.

- ☒ Как должно быть правильно:

Хешировать нужно **введённый пользователем пароль**, который приходит через `user_in.password`:

⚠ **Возможные последствия ошибки:**

- `hashed_password` будет равен `get_password_hash(None)` → это может вызвать исключение или создать некорректный хэш.
- В базе будет сохранён неправильный пароль → при попытке входа `verify_password()` всегда даст `False`.
- ⚠ `login()` содержит `Depends()` внутри тела функции — работать не будет.
- ☒ JWT-токен создается, используется `OAuth2PasswordRequestForm`, есть валидация email — частично выполнена задача [S1] Реализовать CRUD-эндпоинты для регистрации и авторизации пользователей.

5. CRUD для постов

- ☒ Реализованы эндпоинты:
 - `POST /posts`
 - `GET /posts`
 - `GET /posts/{id}`
 - `PUT /posts/{id}`
 - `DELETE /posts/{id}`
- ☒ Проверка прав пользователя на изменение/удаление поста реализована.
- ⚠ `get_current_user` в `posts.py` — заглушка, реальная авторизация не работает — временное решение.
- ☒ Используется SQLAlchemy ORM, `orm_mode=True` в Pydantic — соответствует задаче [S1] Реализовать CRUD-эндпоинты для постов.

6. Миграции Alembic

- ☒ Alembic корректно настроен, `env.py` подключен к `Base.metadata`, `alembic.ini` настроен, миграции работают — задача [S1] Настроить миграции с помощью Alembic выполнена.